

Chapter 1 — ADO.NET Overview & Core Architecture

1. What is ADO.NET?

ADO.NET is the **standard data access technology in the .NET ecosystem** used to interact with relational databases such as:

- SQL Server
- Oracle
- MySQL
- PostgreSQL

It is used in:

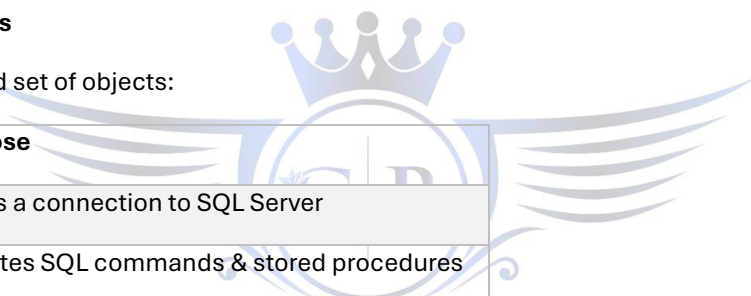
- C#
- .NET Framework
- .NET Core
- ASP.NET / ASP.NET Core
- Console & Desktop applications

Primary role:

To enable applications to **connect, query, manipulate, and manage data stored in databases**, using a structured and reliable API.

2. Core ADO.NET Components

ADO.NET works using a defined set of objects:



Class	Purpose
SqlConnection	Opens a connection to SQL Server
SqlCommand	Executes SQL commands & stored procedures
SqlDataReader	Reads data in a forward-only, read-only manner
SqlDataAdapter	Fills DataSet/DataTable (disconnected mode)
DataSet / DataTable	In-memory data storage

Note:

For SQL Server the namespace is:

System.Data.SqlClient

3. Execution Methods in SqlCommand

Method	Purpose	Returns
ExecuteReader()	Runs SELECT queries	SqlDataReader
ExecuteScalar()	Returns single value (COUNT, SUM, etc.)	object
ExecuteNonQuery()	Executes INSERT / UPDATE / DELETE	int (rows affected)

4. Key Strengths of ADO.NET

- ✓ Fast
 - ✓ Flexible
 - ✓ Supports both **Connected & Disconnected Models**
 - ✓ Works directly with SQL
 - ✓ Suitable for high-performance enterprise apps
-

5. ADO.NET Architecture Models

Connected Architecture

Uses:

- SqlConnection
- SqlCommand
- SqlDataReader

Characteristics:

- ✓ Requires open database connection
 - ✓ Very fast
 - ✓ Forward-only reading
-

Disconnected Architecture

Uses:

- SqlDataAdapter
- DataSet
- DataTable

Characteristics:

- ✓ Data loaded into memory
- ✓ Database connection closes
- ✓ Data manipulation continues offline

Used commonly in:

- Reporting systems
 - Data processing without constant DB connection
-

6. Why Store Connection String in appsettings.json?

Instead of hard-coding connection strings, modern .NET stores them in configuration:

Example:

```
"ConnectionStrings": {  
  "Default": "Data Source=(localdb)\\ProjectModels;Initial Catalog=DemoData_Student;Integrated Security=True;"  
}
```

Benefits

- ✓ Can change at runtime
- ✓ No need to recompile
- ✓ Improves maintainability
- ✓ Supports multiple environments

Retrieving using Dependency Injection:

```
_connectionString = IConfiguration.GetConnectionString("Default");
```

This supports **loosely-coupled enterprise architecture**.

7. POCO Classes and Strong Typing

Instead of printing values directly from `DataReader`, data should be mapped to a POCO:

```
class Product  
{  
  public int Id {get;set;}  
  public string Name {get;set;}  
  public double Price {get;set;}  
  public int Qty {get;set;}  
}
```

Why Mapping is Recommended

- ✓ Strong typing
 - ✓ Clean architecture
 - ✓ Easier maintenance
 - ✓ Supports layering (UI → BLL → DAL → DB)
-

8. ADO.NET Providers

Different databases require different providers.

Database	Provider
SQL Server	System.Data.SqlClient
MySQL	MySqlConnection
ODBC	System.Data.Odbc
OLEDB	System.Data.OleDb

Concepts remain same across providers.

9. SQL Server vs MySQL Object Mapping

SQL Server	MySQL
SqlConnection	MySqlConnection
SqlCommand	MySqlCommand
SqlDataReader	MySqlDataReader

10. Memory & Performance Considerations

Feature	DataReader	DataSet
Architecture	Connected	Disconnected
Direction	Forward-only	Random
Speed	Fastest	Slower
Edit Support	No	Yes
Memory Use	Low	Higher
Use Case	Read-only fast access	Offline CRUD

11. Key Business-Grade Best Practices

- ✓ Use using to ensure connection closure
 - ✓ Never hard-code connection strings
 - ✓ Always use parameterized queries
 - ✓ Handle exceptions
 - ✓ Log critical operations
 - ✓ Close readers & connections properly
-

Chapter 2 — Connections, Commands & Execution Methods

1. SqlConnection — Opening a Database Connection

SqlConnection represents a **physical connection** between your .NET application and SQL Server.

Syntax

```
SqlConnection con = new SqlConnection(_connectionString);
```

```
con.Open();
```

```
// Work with database
```

```
con.Close();
```

Why using is Recommended

```
using(SqlConnection con = new SqlConnection(_connectionString))
```

```
{
```

```
    con.Open();
```

```
}
```

Benefits

- ✓ Ensures automatic disposal
- ✓ Prevents connection leaks
- ✓ Protects SQL Server from exhausting available connections
- ✓ Recommended in enterprise systems



2. SqlCommand — Sending SQL to Database

SqlCommand is responsible for executing SQL statements or stored procedures.

Syntax

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Product", con);
```

It supports:

- SELECT
- INSERT
- UPDATE
- DELETE
- Stored Procedures

3. Command Execution Methods — The Core Three

A. ExecuteReader() — Returns Multiple Rows

Used for:

- ✓ Reading records
- ✓ Forward-only processing
- ✓ High-speed data retrieval

Flow

1. Open connection
2. Create SqlCommand
3. ExecuteReader
4. Loop through rows
5. Close reader
6. Close connection

Example

```
SqlConnection con = new SqlConnection(_connectionString);  
SqlCommand cmd = new SqlCommand("SELECT * FROM Product", con);  
con.Open();  
  
SqlDataReader rdr = cmd.ExecuteReader();  
while(rdr.Read())  
{  
    Console.WriteLine($"{rdr["Id"]} {rdr["Name"]} {rdr["Price"]} {rdr["Qty"]}");  
}
```

Key Characteristics

- ✓ Forward-only
- ✓ Read-only
- ✓ Fastest way to read data
- ✓ Works in **connected mode**

B. ExecuteScalar() — Returns a Single Value

Used for:

- ✓ COUNT
- ✓ SUM
- ✓ AVG
- ✓ MAX / MIN

Example

```
SqlCommand cmd = new SqlCommand("SELECT COUNT(*) FROM Product", con);  
  
int total = (int)cmd.ExecuteScalar();
```

Returns

A single object value

C. ExecuteNonQuery() — Performs CRUD Operations

Used for:

- ✓ INSERT
- ✓ UPDATE
- ✓ DELETE

Example

```
SqlCommand cmd = new SqlCommand("DELETE FROM Product WHERE Id=3", con);  
  
int rows = cmd.ExecuteNonQuery();
```

Return Meaning

Value	Meaning
1	One row modified
0	No rows affected
>1	Multiple rows modified



4. Correct Execution Flow in Real Applications

A standard database operation follows this order:

1. Read connection string from configuration
2. Create SqlConnection
3. Create SqlCommand
4. Assign query or stored procedure
5. Pass parameters
6. Open connection
7. Execute command
8. Process result
9. Close connection safely

This aligns with **SOLID and layered architecture principles**.

5. Dependency Injection — Reading Connection String

In ASP.NET Core:

```
_connectionString = IConfiguration.GetConnectionString("Default");
```

Why This Matters

- ✓ Maintains loose coupling
- ✓ Improves maintainability
- ✓ Enables environment-based configuration

(Example: Dev / QA / Production)

6. Mapping vs Non-Mapping Data Output

Non-Mapped (Direct Print)

Data printed directly from SqlDataReader.

Not recommended for enterprise systems.

Mapped (POCO Model Class) — Recommended

Create a **Product** model:

```
class Product
{
    public int Id {get;set;}
    public string Name {get;set;}
    public double Price {get;set;}
    public int Qty {get;set;}
}
```

Fill List from DataReader

```
List<Product> list = new List<Product>();
```

```
while(rdr.Read())
{
    list.Add(new Product{
        Id = (int)rdr["Id"],
        Name = rdr["Name"].ToString(),
        Price = (double)rdr["Price"],
        Qty = (int)rdr["Qty"]
    });
}
```

Advantages

- ✓ Strong typing
- ✓ Clean code separation

- ✓ Easier debugging
- ✓ Reusable across layers
- ✓ Supports DTOs, ViewModels, API responses

7. CRUD Execution Mapping Summary

Operation	Method	Output
SELECT Multiple	ExecuteReader	List / Rows
SELECT Single	ExecuteReader	Object
INSERT	ExecuteNonQuery	Rows affected
UPDATE	ExecuteNonQuery	Rows affected
DELETE	ExecuteNonQuery	Rows affected
Aggregates (COUNT, SUM)	ExecuteScalar	Single value

8. Error Handling — Try/Catch Recommendation

Wrap DB operations safely:

```
try
{
    using(SqlConnection con = new SqlConnection(_connectionString))
    {
        // DB code
    }
}
catch(Exception ex)
{
    // Log exception
}
```

Logging should be centralized via logging frameworks (e.g., Serilog, NLog).

9. Security & Performance Considerations

- ✓ Ensure connection is always closed
 - ✓ Pooling is managed automatically
 - ✓ Avoid long-running open connections
 - ✓ Prefer minimal data retrieval
 - ✓ Avoid inline SQL concatenation
 - ✓ Use stored procedures or parameters
-

Chapter 3 — Reading Data Using ExecuteReader & Mapping to POCOs

1. Purpose of ExecuteReader

ExecuteReader() is used when you expect the database to return **multiple rows of data**, such as:

```
SELECT * FROM Product
```

```
SELECT Id, Name FROM Employees
```

```
SELECT * FROM Orders WHERE CustomerId = 5
```

It returns a **SqlDataReader**, which acts like a cursor over the result set.

2. Key Characteristics of SqlDataReader

SqlDataReader is:

- ✓ Forward-only
- ✓ Read-only
- ✓ Fastest way to retrieve data
- ✓ Memory efficient
- ✓ Works **only while the database connection is open**

This makes it ideal for **high-performance read operations**.

3. Standard Flow — Reading Multiple Rows

Step-by-step Execution

1. Create connection
2. Create SqlCommand
3. Open connection
4. Call ExecuteReader
5. Loop through rows
6. Extract field values
7. Close reader
8. Close connection

Example

```
SqlConnection con = new SqlConnection(_connectionString);  
SqlCommand cmd = new SqlCommand("SELECT * FROM Product", con);  
con.Open();
```

```
SqlDataReader rdr = cmd.ExecuteReader();
```

```
while(rdr.Read())
{
    Console.WriteLine($"{rdr["Id"]} {rdr["Name"]} {rdr["Price"]} {rdr["Qty"]}");
}
```

4. Meaning of rdr.Read()

Read() returns:

- **true** → if there is a row to read
- **false** → when rows are finished

So the loop terminates automatically.

5. Accessing Column Data

You may read data using:

Column Name

```
rdr["Name"]
```

Column Index

```
rdr[1]
```

Strong Type Methods

```
rdr.GetInt32(0);
```

```
rdr.GetString(1);
```

Strong typing is faster and safer.

6. Using ExecuteReader for Reading a Single Row

When only one record should return:

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Product WHERE Id=@pid", con);
cmd.Parameters.AddWithValue("@pid", id);
```

```
con.Open();
```

```
SqlDataReader rdr = cmd.ExecuteReader();
```

```
Product p = null;
```

```
if(rdr.Read())
{
    p = new Product
    {
        Id = Convert.ToInt32(rdr["Id"]),
```



```
Name = rdr["Name"].ToString(),
Price = Convert.ToDouble(rdr["Price"]),
Qty = Convert.ToInt32(rdr["Qty"])
};
}
```

Important Rule

No loop is needed because only **one row is expected**.

7. When Record Not Found

If Read() returns false → return null.

This is the correct enterprise pattern.

8. Mapping Data to POCOs (Professional Approach)

Rather than printing values directly, map results into domain objects.

POCO Class

```
class Product
{
    public int Id {get;set;}
    public string Name {get;set;}
    public double Price {get;set;}
    public int Qty {get;set;}
}
```



Mapping List from Reader

```
List<Product> list = new List<Product>();
```

```
while(rdr.Read())
{
    list.Add(new Product
    {
        Id = Convert.ToInt32(rdr["Id"]),
        Name = rdr["Name"].ToString(),
        Price = Convert.ToDouble(rdr["Price"]),
        Qty = Convert.ToInt32(rdr["Qty"])
    });
}
```

9. Why Mapping is Essential in Real Systems

- ✓ Enables strong typing
- ✓ Supports layered architecture
- ✓ Eases maintenance
- ✓ Minimizes runtime errors
- ✓ Supports MVC / APIs / Services

POCO objects can now be:

- returned to controllers
- passed to views
- serialized to JSON
- used in business logic

10. Searching Data (LIKE Query)

Stored Procedure

```
CREATE PROCEDURE SP_SearchProduct
```

```
@key NVARCHAR(MAX)
```

```
AS
```

```
BEGIN
```

```
SELECT * FROM Product
```

```
WHERE Name LIKE '%' + @key + '%'
```

```
END
```

DAL Method

```
public List<Product> Search(string keyword)
```

```
{
```

```
    List<Product> list = new List<Product>();
```

```
    using(SqlConnection con = new SqlConnection(_connectionString))
```

```
    {
```

```
        SqlCommand cmd = new SqlCommand("SP_SearchProduct", con);
```

```
        cmd.CommandType = CommandType.StoredProcedure;
```

```
        cmd.Parameters.AddWithValue("@key", keyword);
```

```
        con.Open();
```

```
        SqlDataReader rdr = cmd.ExecuteReader();
```

```
        while (rdr.Read())
```

```
        {
```

```
            list.Add(new Product
```

```
            {
```



```
        Id = Convert.ToInt32(rdr["Id"]),
        Name = rdr["Name"].ToString(),
        Price = Convert.ToDouble(rdr["Price"]),
        Qty = Convert.ToInt32(rdr["Qty"])
    });
}
}
return list;
}
```

This approach:

- ✓ Prevents SQL injection
 - ✓ Improves readability
 - ✓ Separates logic from application code
-

11. Forward-Only Nature — Why It Matters

SqlDataReader only moves **forward**:

Row 1 → Row 2 → Row 3 → ...

You cannot go backward.

You cannot jump to specific record numbers.

This is why it is:

- ✓ fastest
- ✓ lowest memory

Because data is streamed, not stored.



12. Connected Mode Limitation

Connection must **stay open** while reading.

If connection closes, reader becomes invalid.

Therefore:

- Perform only lightweight tasks in loops
 - Never keep UI waiting inside reader loops
 - Avoid business logic execution inside loop
-

13. ExecuteReader vs DataSet (When to Use What)

Use Case	Recommended
Fast read-only display	DataReader
Web API responses	DataReader
Reports needing offline data	DataSet
Editable in-memory data	DataSet
Large streaming	DataReader

14. Error Handling & Safety

Wrap in try-catch:

```
try
{
    using(SqlConnection con = new SqlConnection(_connectionString))
    {
        ...
    }
}
catch(Exception ex)
{
    // Log error
}
```

Logging is critical in production.



Chapter 4 — ExecuteScalar, ExecuteNonQuery & CRUD

1. ExecuteScalar — Returning a Single Value

ExecuteScalar() is used when the query returns exactly **one value**.

Typical examples:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()
- Returning identity / computed value

Example — COUNT Products

```
SqlCommand cmd = new SqlCommand("SELECT COUNT(*) FROM Product", con);
```

```
int total = (int)cmd.ExecuteScalar();
```

Behavior

- Returns **first column of the first row**
- Ignores rest of result set

Use Cases

- ✓ Reports
- ✓ Dashboard metrics
- ✓ Existence checks
- ✓ Validation logic



2. ExecuteNonQuery — Writing Data (INSERT / UPDATE / DELETE)

ExecuteNonQuery() is used when SQL command modifies data but does **not return records**.

Return Meaning

Return Value	Meaning
0	No rows affected
1	One row affected
>1	Multiple rows affected

This helps confirm execution success.

3. INSERT — Adding a Record

Example

```
SqlCommand cmd = new SqlCommand(  
"INSERT INTO Product(Name, Price, Qty) VALUES(@name, @price, @qty)", con);
```

```
cmd.Parameters.AddWithValue("@name", p.Name);
```

```
cmd.Parameters.AddWithValue("@price", p.Price);
```

```
cmd.Parameters.AddWithValue("@qty", p.Qty);
```

```
con.Open();
```

```
int rows = cmd.ExecuteNonQuery();
```

Use ExecuteNonQuery Because

- ✓ No result set is expected
- ✓ Returns affected row count

4. UPDATE — Modifying a Record

Stored Procedure

```
CREATE PROCEDURE SP_UpdateProduct
```

```
@pid INT,
```

```
@pname NVARCHAR(MAX),
```

```
@pprice FLOAT,
```

```
@pqty INT
```

```
AS
```

```
BEGIN
```

```
UPDATE Product
```

```
SET Name = @pname, Price = @pprice, Qty = @pqty
```

```
WHERE Id = @pid
```

```
END
```

DAL Method

```
public int Update(Product p)
```

```
{
```

```
    int rows = 0;
```

```
using(SqlConnection con = new SqlConnection(_connectionString))
{
    SqlCommand cmd = new SqlCommand("SP_UpdateProduct", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@pid", p.Id);
    cmd.Parameters.AddWithValue("@pname", p.Name);
    cmd.Parameters.AddWithValue("@pprice", p.Price);
    cmd.Parameters.AddWithValue("@pqty", p.Qty);

    con.Open();
    rows = cmd.ExecuteNonQuery();
}

return rows;
}
```

Key Advantages

- ✓ Stored procedure abstraction
- ✓ Parameter protection
- ✓ Control logic resides at DB



5. DELETE — Removing a Record

Example

```
SqlCommand cmd = new SqlCommand("DELETE FROM Product WHERE Id=@pid", con);
cmd.Parameters.AddWithValue("@pid", id);
```

```
con.Open();
int rows = cmd.ExecuteNonQuery();
```

Always confirm row count before reporting success.

6. SELECT BY ID — Returning One Record

Method

```
public Product GetById(int id)
{
    Product p = null;

    using(SqlConnection con = new SqlConnection(_connectionString))
    {
        SqlCommand cmd = new SqlCommand(
            "SELECT * FROM Product WHERE Id=@pid", con);

        cmd.Parameters.AddWithValue("@pid", id);

        con.Open();
        SqlDataReader rdr = cmd.ExecuteReader();

        if (rdr.Read())
        {
            p = new Product
            {
                Id = Convert.ToInt32(rdr["Id"]),
                Name = rdr["Name"].ToString(),
                Price = Convert.ToDouble(rdr["Price"]),
                Qty = Convert.ToInt32(rdr["Qty"])
            };
        }
    }
    return p;
}
```



Notes

- ✓ Read() used instead of loop
- ✓ Returns null if not found
- ✓ Clean and deterministic behavior

7. LIKE Query — Searching Data

Stored Procedure

```
CREATE PROCEDURE SP_SearchProduct
    @key NVARCHAR(MAX)
AS
BEGIN
    SELECT * FROM Product
```

```
WHERE Name LIKE '%' + @key + '%'
```

```
END
```

DAL Method

```
public List<Product> Search(string keyword)
{
    List<Product> list = new List<Product>();

    using(SqlConnection con = new SqlConnection(_connectionString))
    {
        SqlCommand cmd = new SqlCommand("SP_SearchProduct", con);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@key", keyword);

        con.Open();
        SqlDataReader rdr = cmd.ExecuteReader();

        while (rdr.Read())
        {
            list.Add(new Product
            {
                Id = Convert.ToInt32(rdr["Id"]),
                Name = rdr["Name"].ToString(),
                Price = Convert.ToDouble(rdr["Price"]),
                Qty = Convert.ToInt32(rdr["Qty"])
            });
        }
    }
    return list;
}
```



8. How These Methods Are Used in Program.cs

```
EmpDAL dal = new EmpDAL(_iconfiguration);

dal.GetList();

dal.AddData(new Product { Name="Laptop", Price=50000, Qty=3 });

dal.Update(new Product { Id=3, Name="Iphone 15", Price=90000, Qty=2 });

dal.Del(5);

dal.GetById(2);

dal.Search("ph");
```

This demonstrates **complete CRUD usage flow**.

9. Difference Summary — One View Table

Operation	ADO.NET Method	Output
SELECT Multiple Rows	ExecuteReader	List / Rows
SELECT One Row	ExecuteReader	Object
INSERT	ExecuteNonQuery	Rows affected
UPDATE	ExecuteNonQuery	Rows affected
DELETE	ExecuteNonQuery	Rows affected
COUNT / SUM / AVG	ExecuteScalar	Single value

This table is essential for exam and interview knowledge.

10. Why using is Mandatory

```
using(SqlConnection con = new SqlConnection(_connectionString))
```

```
{  
    con.Open();  
}
```

Advantages

- ✓ Ensures closure even on exceptions
- ✓ Prevents leaks
- ✓ Prevents pool exhaustion



11. SQL Injection & Parameterized Queries (Very Important)

Vulnerable Code

```
"SELECT * FROM Product WHERE Name LIKE '" + name + "%"
```

If attacker enters:

```
T'; DELETE FROM Product; --
```

The entire table can be deleted.

Safe Alternative

```
SqlCommand cmd =
```

```
new SqlCommand("SELECT * FROM Product WHERE Name LIKE @ProductName", con);
```

```
cmd.Parameters.AddWithValue("@ProductName", name + "%");
```

Benefits

- ✓ Treats input as data
 - ✓ Prevents SQL injection
 - ✓ Type safe
-

Stored Procedures also help prevent injection

```
cmd.CommandType = CommandType.StoredProcedure;
```

12. Golden Security Rules

- ✓ Never concatenate SQL strings
 - ✓ Always use parameterized queries
 - ✓ Validate input
 - ✓ Prefer stored procedures in enterprise systems
-

13. Error Handling & Logging

Always wrap operations:

```
try
{
    ...
}
catch(Exception ex)
{
    // Log exception centrally
}
```

Logging prevents silent failures.



Chapter 5 — Stored Procedures & SQL Injection Prevention

1. What Is a Stored Procedure?

A **Stored Procedure (SP)** is a **pre-compiled SQL program stored inside the database** that can be executed by name.

Key Advantages

- ✓ Improves performance (pre-compiled execution plan)
 - ✓ Centralizes business logic in database
 - ✓ Reduces application code complexity
 - ✓ Supports reusability
 - ✓ Enhances security
 - ✓ Simplifies maintenance
-

2. Why Use Stored Procedures in Enterprise Systems?

Reason	Explanation
Security	Helps prevent SQL injection
Performance	Execution plan is cached
Maintainability	Logic updated without redeploying code
Abstraction	Applications call procedures, not raw SQL
Auditing	Database logging simplifies auditing

Stored procedures become critical as systems scale.

3. Example — Stored Procedure for UPDATE

SQL Server Stored Procedure

```
CREATE PROCEDURE SP_UpdateProduct
```

```
@pid INT,
```

```
@pname NVARCHAR(MAX),
```

```
@pprice FLOAT,
```

```
@pqty INT
```

```
AS
```

```
BEGIN
```

```
UPDATE Product
```

```
SET Name = @pname,
```

```
Price = @pprice,
```

```
Qty = @pqty  
WHERE Id = @pid  
END
```

This updates one product record based on ID.

4. Calling Stored Procedure in ADO.NET

DAL Method

```
public int Update(Product p)  
{  
    int rows = 0;  
  
    using(SqlConnection con = new SqlConnection(_connectionString))  
    {  
        SqlCommand cmd = new SqlCommand("SP_UpdateProduct", con);  
        cmd.CommandType = CommandType.StoredProcedure;  
  
        cmd.Parameters.AddWithValue("@pid", p.Id);  
        cmd.Parameters.AddWithValue("@pname", p.Name);  
        cmd.Parameters.AddWithValue("@pprice", p.Price);  
        cmd.Parameters.AddWithValue("@pqty", p.Qty);  
  
        con.Open();  
        rows = cmd.ExecuteNonQuery();  
    }  
    return rows;  
}
```

Important Notes

- ✓ CommandType must be **StoredProcedure**
 - ✓ Parameters must match SP definition
 - ✓ ExecuteNonQuery returns affected rows
-

5. LIKE Search Stored Procedure

SQL Procedure

```
CREATE PROCEDURE SP_SearchProduct  
  
@key NVARCHAR(MAX)  
  
AS  
  
BEGIN  
  
SELECT * FROM Product  
  
WHERE Name LIKE '%' + @key + '%'
```

END

DAL Implementation

```
public List<Product> Search(string keyword)
{
    List<Product> list = new List<Product>();

    using(SqlConnection con = new SqlConnection(_connectionString))
    {
        SqlCommand cmd = new SqlCommand("SP_SearchProduct", con);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@key", keyword);

        con.Open();
        SqlDataReader rdr = cmd.ExecuteReader();

        while (rdr.Read())
        {
            list.Add(new Product
            {
                Id = Convert.ToInt32(rdr["Id"]),
                Name = rdr["Name"].ToString(),
                Price = Convert.ToDouble(rdr["Price"]),
                Qty = Convert.ToInt32(rdr["Qty"])
            });
        }
    }
    return list;
}
```



6. SQL Injection — A Critical Security Threat

What Is SQL Injection?

It is a hacking technique where user input **alters SQL commands** and compromises the database.

Vulnerable Code Example

```
"SELECT * FROM Product WHERE Name LIKE '" + name + "%"
```

Attacker Input

```
T'; DELETE FROM Product; --
```

This converts your SQL into:

```
SELECT * FROM Product WHERE Name LIKE 'T';
```

```
DELETE FROM Product;
```

```
--%'
```

Meaning:

- ✓ All products deleted
- ✓ No authentication required

This is catastrophic.

7. Parameterized Queries — The Correct Solution

Safe Query

SqlCommand cmd =

```
new SqlCommand("SELECT * FROM Product WHERE Name LIKE @ProductName", connection);
```

```
cmd.Parameters.AddWithValue("@ProductName", name + "%");
```

Why This Is Safe

- ✓ Input treated as data, not SQL
 - ✓ Type converted safely
 - ✓ Prevents query modification
 - ✓ Blocks injection attacks
-

8. Stored Procedures Further Improve Security

```
cmd.CommandType = CommandType.StoredProcedure;
```

Stored procedures:

- ✓ Enforce parameter binding
 - ✓ Can restrict permissions
 - ✓ Reduce surface exposure
-

9. Golden Security Rules (Must-Follow in Production)

- ✓ Never concatenate SQL strings from user input
- ✓ Always use parameterized queries
- ✓ Validate user input
- ✓ Use principle of least privilege
- ✓ Implement database-level role security
- ✓ Avoid inline SQL in presentation layer
- ✓ Prefer stored procedures for critical operations

These principles are essential for **banking, healthcare, ERP, and financial systems.**

10. Best Practice — Always Use using Block

```
using(SqlConnection con = new SqlConnection(_connectionString))  
  
{  
  
    ...  
  
}
```

Ensures

- ✓ Automatic disposal
 - ✓ No connection leaks
 - ✓ Maximum connection pool availability
-

11. Handling NULL Values Correctly

Always check for NULLs:

```
Name = rdr["Name"] == DBNull.Value ? null : rdr["Name"].ToString();
```

Prevents runtime crashes.

12. Error Handling Standard

```
try  
{  
  
}  
  
catch(Exception ex)  
{  
    // Log error with stack trace  
}
```

Production systems **must log exceptions**, not ignore them.

13. Advantages of Parameters & Stored Procedures Combined

- ✓ Defense-in-depth security
- ✓ Robust query handling
- ✓ Reduced risk of system compromise
- ✓ Cleaner code
- ✓ Database maintains control logic
- ✓ Governance & auditing easier

This combination is the **industry standard**.

Chapter 6 — Connected vs Disconnected Architecture (DataReader vs DataSet)

This chapter explains the **two operating models of ADO.NET: Connected and Disconnected**. Understanding this distinction is essential for architecting scalable, high-performance systems.

We will examine:

- Architecture flow
 - Core components
 - Practical use-cases
 - Performance differences
 - Pros and cons
 - When to use which
-

1. Connected Architecture — High-Speed, Real-Time Access

Connected architecture requires the database connection to remain **open during the entire data operation**.

ADO.NET Classes Used

- SqlConnection
 - SqlCommand
 - SqlDataReader
-



2. Flow of Connected Architecture

1. Open database connection
2. Send SQL command
3. Execute and fetch results
4. Process data
5. Close connection

This is **fast and streaming-based**.

3. Key Characteristics

- ✓ Forward-only reading
 - ✓ Read-only
 - ✓ Minimum memory usage
 - ✓ Highest performance
 - ✓ Real-time streaming of data
-

4. Example — Connected Mode Using SqlDataReader

```
SqlConnection con = new SqlConnection(_connectionString);
SqlCommand cmd = new SqlCommand("SELECT * FROM Product", con);

con.Open();

SqlDataReader rdr = cmd.ExecuteReader();

while(rdr.Read())
{
    Console.WriteLine($"{rdr["Id"]} {rdr["Name"]} {rdr["Price"]} {rdr["Qty"]}");
}
```

5. Limitations of Connected Architecture

- ✗ Connection must remain open
 - ✗ Does not support in-memory editing
 - ✗ Forward-only navigation
 - ✗ Dependent on active DB connectivity
-

6. Best Use-Cases for Connected Model

- ✓ High-performance read operations
- ✓ Large data streaming
- ✓ Web API responses
- ✓ Reports where modification is not required
- ✓ Data display grids
- ✓ Dashboard queries

It is the **default choice for real-time operations**.

7. Disconnected Architecture — In-Memory Data Handling

Disconnected architecture allows applications to **retrieve data from the database, store it in memory, and then close the connection**.

Work continues **without requiring an active connection**.

8. Components Used

- SqlDataAdapter
 - DataTable
 - DataSet
-

9. Flow of Disconnected Architecture

1. Open connection
2. Execute SELECT via DataAdapter
3. Data loaded into DataSet / DataTable
4. Connection closes
5. Further work continues in memory

This means the database is **not locked or engaged** while working with data.

10. Example — Disconnected Mode

```
SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Product", con);
```

```
DataSet ds = new DataSet();
```

```
da.Fill(ds);
```

```
DataTable dt = ds.Tables[0];
```

11. Characteristics

- ✓ Supports offline mode
 - ✓ Data stored locally
 - ✓ Random access
 - ✓ Supports CRUD in memory
 - ✓ Supports binding to UI controls
-

12. Limitations

- ✗ Higher memory usage
 - ✗ More overhead
 - ✗ Slightly slower compared to DataReader
 - ✗ Requires sync logic to update database
-

13. DataReader vs DataSet — Examination-Ready Comparison

Feature	DataReader	DataSet
Architecture	Connected	Disconnected
Direction	Forward-only	Random
Speed	Fastest	Slower
Edit Support	No	Yes
Memory	Low	Higher
Storage	Stream	In-memory
Use Case	Read-only data	Offline CRUD & Reporting

This table is **critical for both interviews and exams**.

14. Real-World Use Case Scenarios

Use DataReader When

- ✓ Reading large data fast
- ✓ Display-only operations
- ✓ Performance is priority
- ✓ Memory must be optimized



Use DataSet / DataTable When

- ✓ Data should persist offline
 - ✓ Need editable in-memory data
 - ✓ Data binding required
 - ✓ Multiple tables must be joined
 - ✓ Reports need static snapshots
-

15. Example — Updating Data via Disconnected Mode

```
SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Product", con);  
SqlCommandBuilder cb = new SqlCommandBuilder(da);
```

```
DataSet ds = new DataSet();  
da.Fill(ds);
```

```
ds.Tables[0].Rows[0]["Name"] = "UpdatedName";
```

```
da.Update(ds);
```

This updates the database using in-memory changes.

16. When to Avoid Disconnected Model

Avoid when:

- ✗ System is memory-sensitive
 - ✗ Live streaming is required
 - ✗ Data size is very large
-

17. Why ADO.NET Provides Both Models

Because **different business scenarios require different execution strategies.**

Connected Model = Speed Focus

Disconnected Model = Flexibility Focus

18. Best Practices

- ✓ Use DataReader for high-speed reads
 - ✓ Use DataSet for offline & UI binding
 - ✓ Release connections quickly
 - ✓ Avoid loading huge tables into memory
 - ✓ Prefer paging logic for large datasets
-



Chapter 7 — Transactions & ACID — Ensuring Data Integrity in ADO.NET

1. What Is a Transaction?

A **transaction** is a logical unit of work consisting of one or more database operations that must either:

- ✓ **All succeed together**, or
- ✓ **All fail and roll back**

There is **no partial success allowed**.

2. Why Are Transactions Needed?

Consider a **bank transfer**:

- Debit account A
- Credit account B

If only debit succeeds and credit fails:

- ✓ Money disappears
- ✓ Data becomes corrupted

Therefore both operations must behave as **one atomic unit**.

3. ACID Properties — Foundation of Transactions

Transactions in databases follow **ACID properties**:

Property	Meaning
Atomicity	All or none
Consistency	Data remains valid before & after transaction
Isolation	Transactions do not affect each other
Durability	Data remains safe even after failure

These principles make database systems **trustworthy and reliable**.

4. Transaction Support in ADO.NET

ADO.NET supports transactions using:

- `SqlTransaction`
- `BeginTransaction()`
- `Commit()`
- `Rollback()`

5. Transaction Flow

1. Open connection
 2. Begin transaction
 3. Execute commands
 4. If all succeed → Commit
 5. If any fail → Rollback
-

6. Example — Transaction in ADO.NET

```
using(SqlConnection con = new SqlConnection(_connectionString))
{
    con.Open();

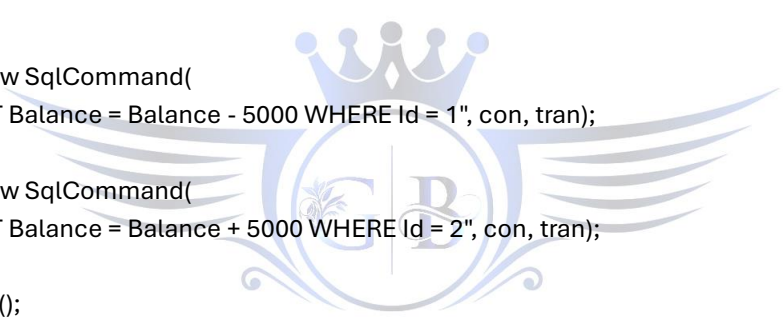
    SqlTransaction tran = con.BeginTransaction();

    try
    {
        SqlCommand cmd1 = new SqlCommand(
            "UPDATE Accounts SET Balance = Balance - 5000 WHERE Id = 1", con, tran);

        SqlCommand cmd2 = new SqlCommand(
            "UPDATE Accounts SET Balance = Balance + 5000 WHERE Id = 2", con, tran);

        cmd1.ExecuteNonQuery();
        cmd2.ExecuteNonQuery();

        tran.Commit();
    }
    catch
    {
        tran.Rollback();
    }
}
```



7. Explanation of the Above Code

- ✓ Both queries are executed under a single transaction
 - ✓ If any command fails, rollback occurs
 - ✓ Database reverts to previous state
 - ✓ Ensures no partial update
-

8. When to Use Transactions

Use transactions when:

- ✓ Multiple related updates occur
- ✓ Data must never become inconsistent
- ✓ Business logic requires all-or-nothing
- ✓ High-value data operations occur

Examples:

- Bank transfers
- Order processing
- Stock updates
- Payroll
- Ledger posting
- Inventory updates

9. Database Integrity Without Transactions — Risks

Without transactions, the system may experience:

- ✗ Dirty writes
- ✗ Inconsistent balances
- ✗ Lost updates
- ✗ Financial loss
- ✗ Legal compliance violations



Therefore, **transactions are non-negotiable in real systems.**

10. Isolation Levels (Conceptual Awareness)

Isolation levels control visibility of data between transactions:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

Higher isolation → more safety but lower concurrency.

11. Best Practices for Transactions

- ✓ Keep transactions short
- ✓ Avoid user input inside transaction block

- ✓ Log failures
 - ✓ Use error handling
 - ✓ Use stored procedures when possible
 - ✓ Release locks quickly
-

12. ACID in One Picture

- **Atomicity** → All or nothing
- **Consistency** → Valid database state
- **Isolation** → Parallel safety
- **Durability** → Crash-proof data

Together they **guarantee trust in data systems**.



Chapter 8 — Best Practices, Dependency Injection & Configuration (appsettings.json + DAL Layer Design)

1. Why We Avoid Hard-Coding the Connection String

A **connection string must never be hard-coded** inside application code because:

- ✓ It may change across environments (Dev / QA / Prod)
- ✓ It can expose sensitive credentials
- ✓ Updating code requires recompilation
- ✓ Maintenance becomes painful
- ✓ Violates separation of concerns

Instead, we store it in **appsettings.json**.

2. Connection String in appsettings.json

Example

```
{  
  "ConnectionStrings": {  
    "Default": "Data Source=(localdb)\\ProjectModels;Initial Catalog=DemoData_Student;Integrated Security=True;"  
  }  
}
```

This file is **external configuration**, not compiled code.

3. Reading the Connection String Using Dependency Injection (DI)

In ASP.NET Core, IConfiguration retrieves configuration values.

Inside DAL Class Constructor

```
private readonly string _connectionString;  
  
public EmpDAL(IConfiguration iconfiguration)  
{  
    _connectionString = iconfiguration.GetConnectionString("Default");  
}
```

This Achieves

- ✓ Loose coupling
- ✓ Easier testing
- ✓ Clear separation of concerns
- ✓ Configurable environments

This is **constructor-based dependency injection**, which is the recommended pattern.

4. Why Dependency Injection Is Essential

Dependency Injection:

- ✓ Avoids tight dependency on configuration source
- ✓ Improves maintainability
- ✓ Enables inversion of control
- ✓ Supports unit testing
- ✓ Reduces hidden dependencies

This aligns with **SOLID architecture principles**.

5. Correct Layered Architecture

A professional system does not directly access the database from UI. Instead, it follows **N-Tier structure**:

Typical Layers

1. **Presentation Layer (UI / MVC / API)**
2. **Business Logic Layer (BLL)**
3. **Data Access Layer (DAL)**
4. **Database**

6. DAL (Data Access Layer) Role

DAL is responsible for:

- ✓ Database connectivity
 - ✓ CRUD operations
 - ✓ Mapping database results to models
 - ✓ Returning POCO objects
 - ✓ Shielding rest of app from SQL logic
-

7. Using using Block — Mandatory Best Practice

```
using(SqlConnection con = new SqlConnection(_connectionString))
{
    con.Open();
}
```

Why?

- ✓ Ensures automatic disposal
- ✓ Prevents connection leaks
- ✓ Protects SQL Server from pool exhaustion
- ✓ Works even when exceptions occur

This is critical for **long-running production systems**.

8. Global Best Practices for ADO.NET

A. Use Parameterized Queries Always

- ✓ Prevents SQL injection
- ✓ Type-safe values
- ✓ Cleaner query logic

B. Never Concatenate SQL Strings

This is unsafe and unacceptable in production.

C. Close Connections Quickly

Only keep connection open for:

- ✓ ExecuteReader duration
- ✓ ExecuteNonQuery duration

Never hold it longer.

D. Wrap Database Calls in try-catch

- ✓ Log errors
- ✓ Prevent crashes
- ✓ Maintain reliability

E. Log Database Exceptions

Logging ensures auditability and troubleshooting.

F. Avoid SELECT *

- ✓ Retrieve only required columns
- ✓ Improve performance

G. Use Stored Procedures for Business-Critical Logic

- ✓ Improves security
 - ✓ Improves performance
-

H. Validate User Input

- ✓ sanitize
 - ✓ type-check
 - ✓ length-check
-

9. POCO Models — Strong Typing Standard

Example Product Class

```
class Product
{
    public int Id {get;set;}
    public string Name {get;set;}
    public double Price {get;set;}
    public int Qty {get;set;}
}
```

This ensures:

- ✓ Type safety
 - ✓ Model consistency
 - ✓ Clear contracts between layers
-

10. Using Configuration with DI — Advantages Summary

Benefit	Explanation
Maintainability	Change DB without code modification
Security	Sensitive info not exposed in code
Environment Flexibility	Multiple DB configs possible
Testability	DAL independent from configuration source
Loose Coupling	No hard dependency on config location

11. Example — Calling DAL from Program / Controller

```
EmpDAL dal = new EmpDAL(_iconfiguration);
```

```
dal.GetList();
dal.AddData(new Product { Name="Laptop", Price=50000, Qty=3 });
dal.Update(new Product { Id=3, Name="Iphone 15", Price=90000, Qty=2 });
dal.Del(5);
dal.GetById(2);
dal.Search("ph");
```

The DAL handles **all ADO.NET logic internally**.

12. Exception Handling Strategy

Use structured try-catch:

```
try
{
    ...
}
catch(Exception ex)
{
    // Log exception using logging framework
}
```

Recommended logging frameworks:

- ✓ Serilog
 - ✓ NLog
 - ✓ Microsoft.Extensions.Logging
-

13. Why Logging Matters

- ✓ Helps in root cause analysis
 - ✓ Supports diagnostics
 - ✓ Creates audit trails
 - ✓ Critical for compliance industries
-

14. Golden Rules Summary

- ✓ Always use using with SqlConnection
- ✓ Never hard-code connection strings
- ✓ Always use parameterized queries
- ✓ Close connection immediately after use
- ✓ Validate inputs
- ✓ Map results to POCOs
- ✓ Use DI for reading connection string
- ✓ Wrap DB operations in try-catch
- ✓ Log exceptions
- ✓ Follow layered architecture

These practices define **enterprise-quality ADO.NET applications**.

1. What Is a Data Provider in ADO.NET?

A **data provider** is a set of classes that enables communication between .NET and a specific database engine.

Each provider contains its own versions of:

- Connection class
- Command class
- DataReader class
- DataAdapter class

However, **concepts and usage patterns remain the same.**

2. Common ADO.NET Providers

Database	Provider Namespace
SQL Server	System.Data.SqlClient
MySQL	MySqlConnection
ODBC	System.Data.Odbc
OLEDB	System.Data.OleDb



3. SQL Server Provider (Most Common)

Namespace:

System.Data.SqlClient

Key classes:

SQL Server Class	Purpose
SqlConnection	Database connection
SqlCommand	Execute SQL / SP
SqlDataReader	Read streaming data
SqlDataAdapter	Fill DataSet / DataTable

All examples in earlier chapters used this provider.

4. MySQL Provider — MySqlConnection

To work with MySQL, use the **MySqlConnection** library.

Equivalent classes:

SQL Server	MySQL
SqlConnection	MySqlConnection
SqlCommand	MySqlCommand
SqlDataReader	MySqlDataReader
SqlDataAdapter	MySqlDataAdapter

Other than class names, **logic remains identical**.

5. ODBC Provider — System.Data.Odbc

Used for:

- ✓ Legacy systems
- ✓ Cross-platform database access
- ✓ Systems where DB vendor driver is ODBC-compliant

Equivalent classes:

Purpose	Class
Connection	OdbcConnection
Command	OdbcCommand
Reader	OdbcDataReader



6. OLEDB Provider — System.Data.OleDb

Used for:

- ✓ MS Access
- ✓ Excel
- ✓ Legacy COM-based databases

Class hierarchy mirrors others.

7. Key Insight — ADO.NET Programming Model Is Consistent

Regardless of provider:

- Open connection
- Create command
- Add parameters
- Execute

- Read / update data

So knowledge gained for SQL Server **directly applies to other databases.**

8. Provider Selection Guidelines

Requirement	Choose
Microsoft SQL Server	System.Data.SqlClient
MySQL	MySqlConnection
Multiple DB compatibility	ODBC
MS Access / Excel / Legacy COM	OleDb

9. Concepts Remain the Same Across Providers

- ✓ ExecuteReader
- ✓ ExecuteScalar
- ✓ ExecuteNonQuery
- ✓ Parameterization
- ✓ Transactions
- ✓ Stored Procedures (where supported)
- ✓ DataReader vs DataSet design decision

This makes ADO.NET **flexible and extendable.**

